



Politechnika Wrocławska

Projekt 1. Badanie wydajności wybranych algorytmów sortowania z uwzględnieniem złożoności czasowej oraz obliczeniowej dla różnych rozmiarów danych, wygenerowanych losowo

Programowanie i analiza algorytmów

Michał Suchoń (nr indeksu: 284088)

ISA

Spis treści

1	Wprowadzenie	3
1.1	MergeSort sortowanie przez scalanie	3
1.1.1	Teoretyczne złożoności obliczeniowe	3
1.2	QuickSort sortowanie szybkie	4
1.2.1	Teoretyczne złożoności obliczeniowe	5
1.3	IntroSort sortowanie introspektywne	5
1.3.1	Teoretyczne złożoności obliczeniowe	5
2	Wyniki pomiarów	6
3	Analiza wyników, wnioski	8
4	Wykresy	9
5	Literatura	13

1 Wprowadzenie

Celem projektu jest przeprowadzenie badań trzech algorytmów sortowania, sprawdzenie ich złożoności obliczeniowych dla różnego stopnia wstępnie posortowanych danych, a także dla różnych rozmiarów danych. Badania przeprowadzono dla algorytmów MergeSort (sortowanie przez scalanie), QuickSort (sortowanie szybkie) oraz IntroSort (sortowanie introspektywne), wykorzystujące QuickSort oraz HeapSort (sortowanie przez kopcowanie). Wyniki zostały zaprezentowane w postaci tabel z uśrednionymi wynikami, a także poprzez wykresy, przedstawiające uzyskane złożoności obliczeniowe.

Zakres danych obejmuje następujące rozmiary:

$$n \in \{10000; 50000; 100000; 500000; 1000000\} \quad (1)$$

Wszystkie dane obejmują losowo wygenerowane liczby z zakresu $x \in [0, 100]$ za pośrednictwem funkcji `high_resolution_clock()` z biblioteki `chrono` na bazie silnika liczb losowych `mt19937`, zaś zakresy losowe liczb całkowitych uzyskano w sposób równomierny poprzez `uniform_int_distribution()`.

Dążąc do zachowania unikatowości eksperymentu i uniknięcia wpływów szumów podczas wyznaczania czasu wykonania procesu sortowania, pomiarów dokonano 100 razy dla każdego rozmiaru n , uwzględnionego powyżej.

Do pomiaru czasu (w milisekundach) wykorzystano autorską funkcję `measureExecutionTimeMs()`, która jest szablonową funkcją wyższego rzędu (*higher-order function*), przyjmującą jako argument docelową funkcję sortującą, która wywołana została poprzez instrukcję `std::invoke`.

Wyniki zostały zapisane do plików z rozszerzeniem CSV i są dostępne w kodzie źródłowym, w folderze `build`.

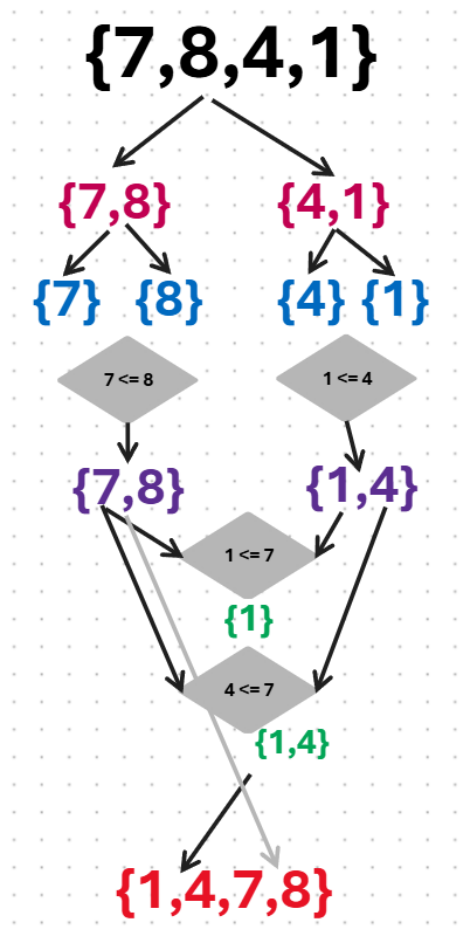
1.1 MergeSort | sortowanie przez scalanie

Algorytm, który wykorzystuje strategię *dziel i rządz*. Poprzez podział należy rozumieć wybór dowolnego elementu ze zbioru (zazwyczaj wartość środkową), a następnie wyznaczenie dwóch podzbiorów: po lewej oraz prawej stronie danego elementu (z włączeniem go do dowolnego z tych dwóch podzbiorów). Proces podziału powtarza się do momentu uzyskania podzbiorów jednoelementowych uniemożliwiających dalszy podział. Następnie, elementy zostają scalone do jednego podzbioru z równoczesnym ich porównaniem i ustawieniem zgodnie z określonym wcześniej porządkiem (rosnącym bądź malejącym). Dobór kolejności scalania odpowiednich elementów oparty jest o znalezienie wartości minimalnych oraz ich porównanie (celem wyboru tego o mniejszej wartości). Kolejne etapy zakładają powtarzanie tych procesów aż do momentu uzyskania jednolitego pełnego zbioru, z posortowaną zawartością. Na potrzeby elastycznego wyboru mniejszej wartości spośród dwóch porównywanych, często w algorytmach stosuje się tzw. `bufor` (*ang. buffer*) do tymczasowego zapisu danego elementu podzbioru ze złożonością pamięciową $O(n)$. Przykładowy proces sortowania z wykorzystaniem mechanizmu *dziel i rządz* przedstawiono na Rysunku 1.

1.1.1 Teoretyczne złożoności obliczeniowe

- Pesymistyczna: $O(n \log n)$
- Średnia: $O(n \log n)$
- Optymistyczna: $O(n \log n)$

Złożoności te wynikają z faktu, iż algorytm każdorazowo dzieli sortowaną strukturę na podstruktury o poziomach, równych $\log_2 n$, gdzie w każdym dokonywane jest $O(n)$ operacji scalania.



Rysunek 1: Schemat działania algorytmu MergeSort dla 4 elementów - opracowanie własne

1.2 QuickSort | sortowanie szybkie

Podobnie do MergeSort, również w sortowaniu szybkim zachowana została zasada *dziel i rządź* z tą różnicą, iż w tym sortowaniu wartością-punktem odniesienia jest *pivot*, który zazwyczaj ustawia się na ostatni element, choć wydajniejszym sposobem jest zastosowanie tzw. *Median Of Three* (mediana trzech punktów) - elementu pierwszego (*low*), środkowego (*mid*) oraz końcowego (*high*). Następnie, dokonywany jest podział na 3 podzbiory zgodnie z tzw. *3-way partitioning* nazywaną również *Dutch National Flag partition*. Partycjonowanie zgodnie z tą zasadą zakłada wybór trzech podzbiorów: elementy mniejsze od *pivota*, równe wartości *pivota* oraz większe od *pivota*.

Na potrzeby algorytmu, zaimplementowane zostały trzy wskaźniki symbolizujące zakresy każdego z wymienionych podzbiorów:

- *lt* - *lower than...*; elementy mniejsze od *pivota*
- *i* - *item*; elementy równe *pivotu*
- *gt* - *greater than...*; elementy większe od *pivota*

W dalszej kolejności, dokonywana jest seria porównań do momentu, dopóki wartość ze wskaźnika *i* jest mniejsza od wartości ze wskaźnika *gt*.

W algorytmie zaimplementowane zostało porównanie, zgodnie z którym sprawdzane jest poprzez funkcję `comp(a,b)`, czy element po lewej stronie jest mniejszy od elementu po prawej stronie ($a < b$) - funkcja `std::less()`. Jeżeli tak postawiony warunek dla wartości wskaźnika *i* po lewej stronie (*a*) oraz *pivota* po prawej stronie (*b*) jest prawdziwy, dokonywana jest zamiana miejscami elementów ze wskaźników *lt* oraz *i*, po czym oba te wskaźniki przesuwane są w prawą stronę. W analogiczny sposób, jeżeli zaś warunek ten nie zostanie spełniony - porównuje się wartości wskaźników zamieniając je miejscami, tzn. `comp(pivot, *i)` i wówczas gdy warunek zwróci wartość `true`: zamieniane są miejscami

wartości ze wskaźników *i* oraz *gt*. Następnie, wskaźnik *gt* przesuwany jest w lewą stronę. Alternatywnie, gdy elementy są takie same (oba poprzednie warunki nie zostały spełnione), przesuwany jest wskaźnik *i* w prawą stronę.

1.2.1 Teoretyczne złożoności obliczeniowe

- Pesymistyczna: $O(n^2)$
- Średnia: $O(n \log n)$
- Optymistyczna: $O(n \log n)$

Pesymistyczna złożoność obliczeniowa zachodzi w sytuacji, gdy podział elementów na podzbiory jest dokonywany w sposób niesymetryczny (wielkości podzbiorów różnią się), np. gdy każdorazowo zostanie wybrany element o największej bądź najmniejszej wartości.

1.3 IntroSort | sortowanie introspektywne

To algorytm, złożony z kilku rodzajów sortowań: szybkiego (QuickSort), przez kopcowanie (HeapSort), a także przez wstawienie (InsertSort). Podobnie do pozostałych omówionych algorytmów, wykorzystuje strategię *dziel i rządź* przy jednoczesnej dbałości o poziom rekurencyjności, tzn. gdy wyniesie on 0, algorytm przełącza się z początkowego działania jako QuickSort na sortowanie przez kopcowanie (HeapSort). Ponadto, dla niewielkich struktur danych możliwe jest także zmniejszenie złożoności obliczeniowej poprzez automatyczne przełączenie na sortowanie przez wstawianie (InsertSort), co z uwagi na specyfikę projektu zostało pominięte (nie odgrywa to szczególnej roli dla testów jednostkowych, zaś sam cel badania zakłada wykorzystanie dużych zbiorów danych).

Ustalenie poziomu głębokości rekurencji jest wyznaczane poprzez wzór:

$$depthLimit = 2 \cdot \log_2(n) \quad (2)$$

Jeżeli wartość ta zostanie osiągnięta, tzn. algorytm dojdzie do stanu $depthLimit = 0$ w związku ze stopniowym pogłębianiem się poziomu rekurencji i wynikającym z tego odejmowaniem od zadanej wartości początkowej, algorytm przełączy się z sortowania mechanizmem QuickSort na HeapSort, ograniczając w ten sposób zwiększanie złożoności obliczeniowej.

Na potrzeby badań i w celach przykładu, QuickSort zastosowany w niniejszym sortowaniu introspektywnym wykorzystuje wariant algorytmu partycjonowania o nazwie *Lomuto*. Zakłada on ustawienie pivotu na ostatni element struktury, a następnie porównywanie wartości z pivotem w taki sposób, aby wartości mniejsze od pivotu znalazły się po jego lewej stronie, zaś większe - po prawej, z jednoczesnym każdorazowym kopiowaniem struktury i sortowaniem na niej, a następnie nadpisywanie oryginalnej kopią.

Różnica pomiędzy algorytmami *Lomuto* od *3-way partitioning* wynika z ilości wykorzystanych wskaźników: w pierwszym przypadku wykorzystywane są wyłącznie dwa (*i* oraz *j*), zaś w drugim trzy, tzn. (*lt*, *i*, *gt*), co usprawnia proces sortowania. W związku z niniejszym, sortowanie introspektywne w uzyskanych wynikach może wypaść gorzej (mieć dłuższy czas wykonania), lecz zabieg ten jest zamierzony celem podkreślenia wpływu doboru metod partycjonowania danych względem dwóch odrębnych algorytmów sortowania.

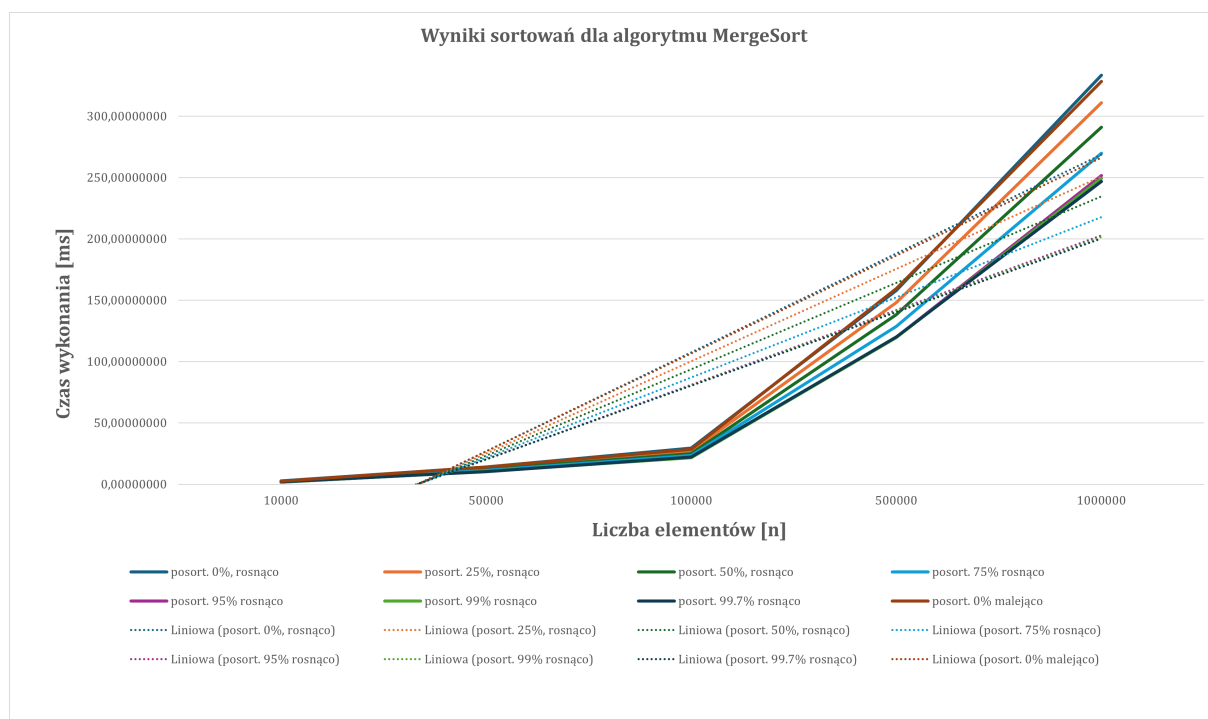
1.3.1 Teoretyczne złożoności obliczeniowe

- Pesymistyczna: $O(n \log n)$
- Średnia: $O(n \log n)$
- Optymistyczna: $O(n \log n)$

2 Wyniki pomiarów

rozmiar	ŚREDNIE WYNIKI SORTOWAŃ DLA MERGE SORT PRZY POSORTOWANYCH n% ELEMENTÓW							MALEJĄCO
	0%	25%	50%	75%	95%	99%	99.7%	
10000	2,67522690	2,32300240	2,13545993	1,99358909	1,91013698	1,89514609	1,92930589	2,51633616
50000	14,16185772	12,99808215	12,16375847	11,43162223	10,65683102	10,49272083	10,42431686	14,09457033
100000	29,57136819	27,18904162	24,93869640	23,42771865	21,94493091	21,62734971	22,28955633	28,58053334
500000	158,03909660	148,19596537	138,47103868	128,80269592	119,92792010	119,66490868	120,22735189	159,45907355
1000000	333,56441228	310,96046798	291,17521932	269,94883102	251,65844605	249,52123182	246,99594370	328,55331950

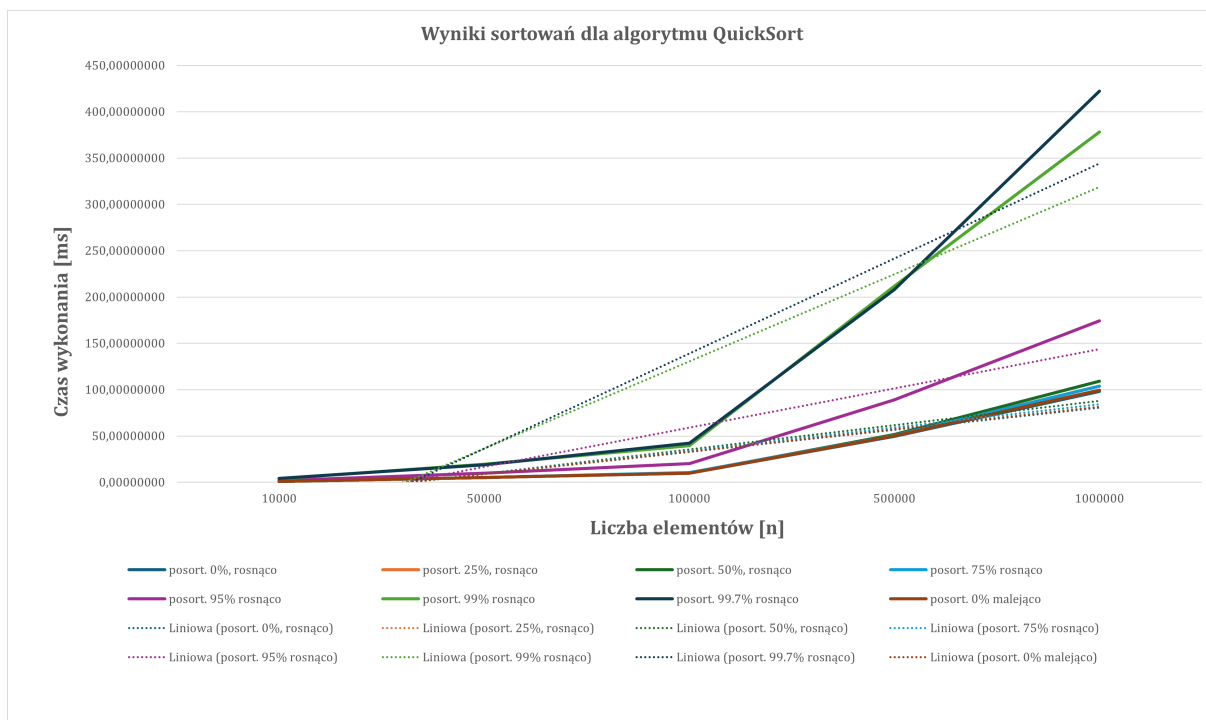
Rysunek 2: Uśrednione wyniki 100 prób sortowania dla algorytmu MergeSort [wyrażone w milisekundach (ms)]



Rysunek 3: Zbiorczy wykres przedstawiający uśrednione wyniki sortowań 100 próbek dla MergeSort w zależności od stopnia (w %) wstępnie przesortowanych danych

rozmiar	ŚREDNIE WYNIKI SORTOWAŃ DLA QUICK SORT PRZY POSORTOWANYCH n% ELEMENTÓW							MALEJĄCO
	0%	25%	50%	75%	95%	99%	99.7%	
10000	1,04823161	1,06298298	1,10956585	1,09904189	1,96465500	3,82938801	4,14544355	1,05715902
50000	5,24605326	5,14125440	5,13010495	5,22890698	9,87131308	19,64946409	18,94187210	5,09806526
100000	10,02549685	10,13821967	10,36405924	10,51639699	20,22251718	39,62342120	42,35388249	10,05930102
500000	49,68486114	50,62584584	51,55527087	50,74042858	89,07304668	211,64223684	208,16052411	50,34827429
1000000	98,19411587	103,95081219	109,13753660	103,70275372	174,46094137	378,20829726	422,34965957	99,40544972

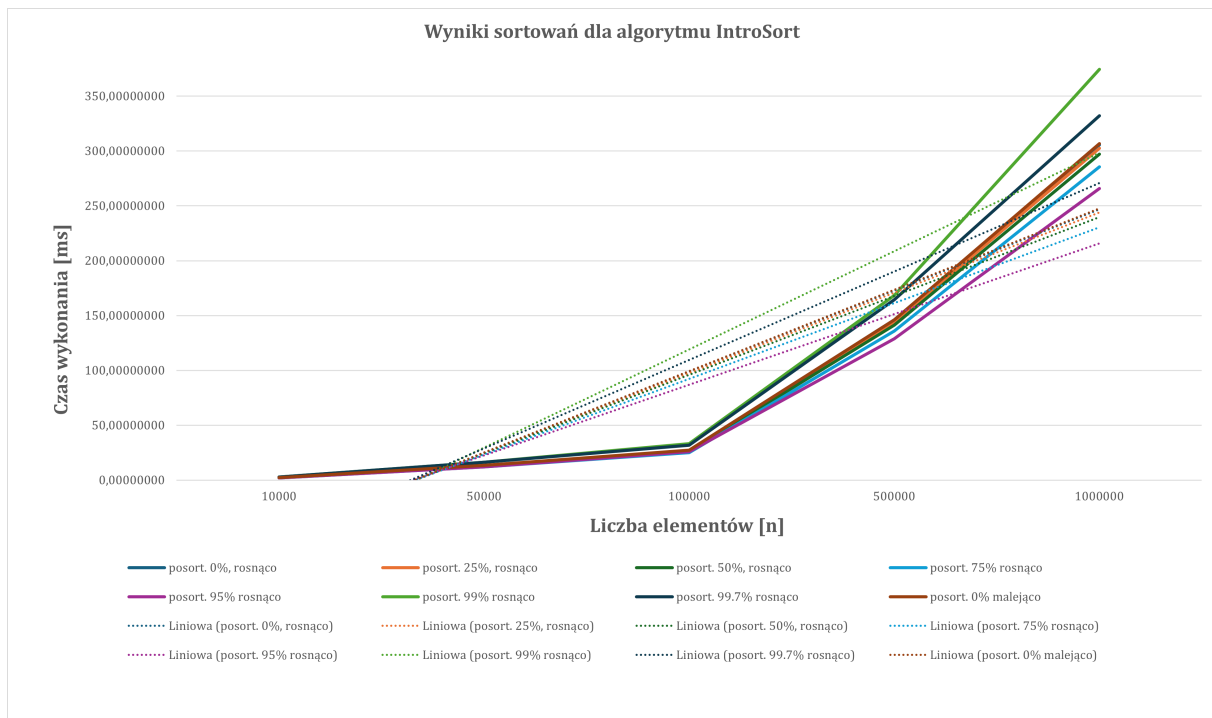
Rysunek 4: Uśrednione wyniki 100 prób sortowania dla algorytmu QuickSort [wyrażone w milisekundach (ms)]



Rysunek 5: Zbiorczy wykres przedstawiający uśrednione wyniki sortowań 100 próbek dla QuickSort w zależności od stopnia (w %) wstępnie posortowanych danych

rozmiar	ŚREDNIE WYNIKI SORTOWAŃ DLA INTRO SORT PRZY POSORTOWANYCH n% ELEMENTÓW							MALEJĄCO
	0%	25%	50%	75%	95%	99%	99.7%	
10000	2,25980049	2,19904915	2,17040774	2,12468091	2,09477800	2,94597012	2,95900684	2,31698746
50000	12,81967088	12,92118696	12,44694621	12,12673105	12,11935137	16,45975718	16,28322601	13,53377431
100000	26,98580624	26,78747730	26,20930473	25,20835914	25,85153556	33,38330213	31,95868564	27,41766857
500000	146,03237753	144,05394899	141,22783563	135,93658952	129,01149528	168,46188160	164,55284154	146,24642619
1000000	305,66709968	302,45823341	297,18024161	285,61602652	265,74323277	374,39802919	332,09988939	306,64117753

Rysunek 6: Uśrednione wyniki 100 prób sortowania dla algorytmu IntroSort [wyrażone w milisekundach (ms)]



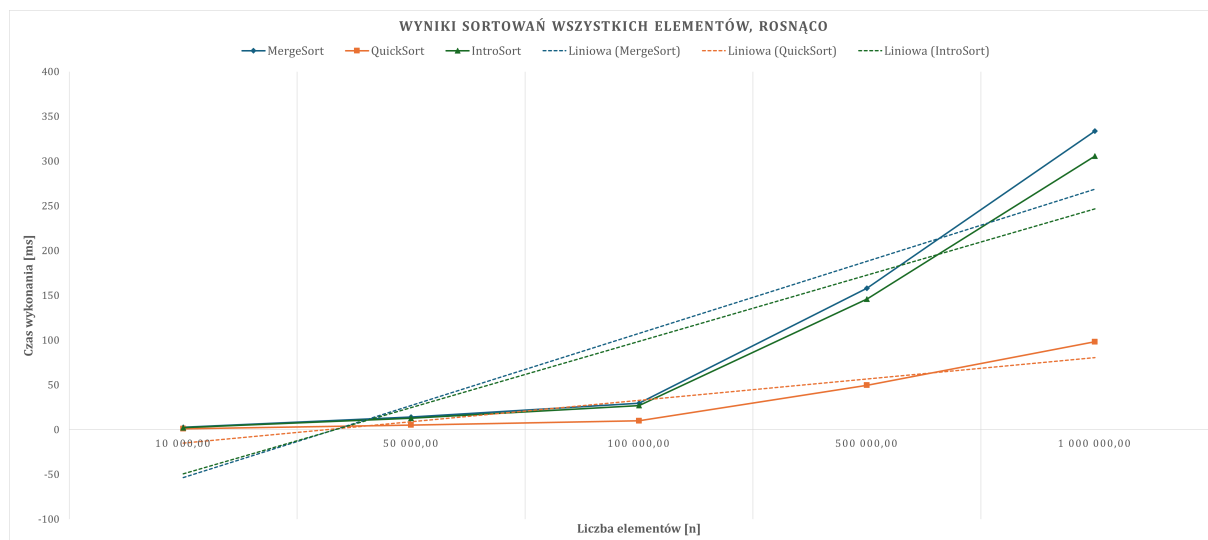
Rysunek 7: Zbiorczy wykres przedstawiający uśrednione wyniki sortowań 100 próbek dla IntroSort w zależności od stopnia (w %) wstępnie posortowanych danych

Wykresy zestawiające różne algorytmy sortujące w zależności od określonej ilości wstępnie posortowanych elementów zostały zamieszczone na końcu niniejszego sprawozdania.

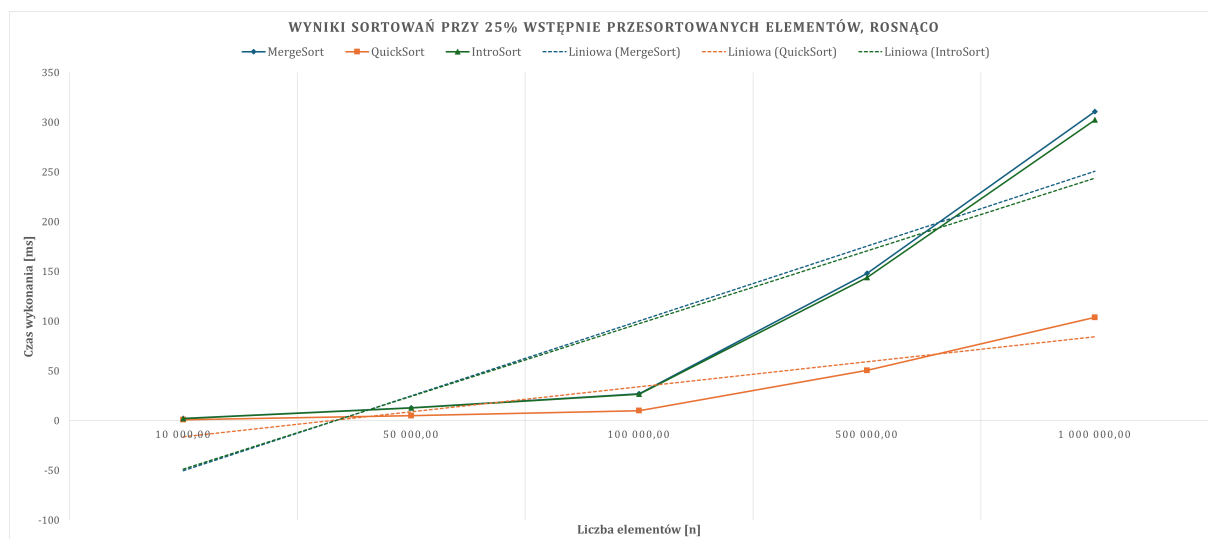
3 Analiza wyników, wnioski

- Trudno jednoznacznie wskazać, jaką złożoność obliczeniową prezentują niniejszym uzyskane wyniki pomiarów. Zestawiając je jednak z wykresami teoretycznych złożoności, wykresy te kształtem oscylują między złożonością $O(n \log n)$, a $O(n^2)$, co prawdopodobnie wynika z równoległego działania innych procesów systemowych, mających wpływ na zaszumienie docelowego czasu wykonania dla poszczególnych rozmiarów danych.
- Najlepszą skuteczność dla danych całkowicie nieposortowanych (zarówno rosnąco jak i malejąco) posiada QuickSort, który osiągnął najmniejszy czas wykonania algorytmu niezależnie od ilości elementów w zestawieniu z pozostałymi algorytmami. Największe wartości czasu wykonania posiada MergeSort, choć niewiele mniej osiągnęło sortowanie introspektywne (IntroSort), co wynika z zastosowania wariantu Lomuto dla partycjonowania danych na podzbiory.
- W większości przypadków także najszybszy czas wykonania posiada QuickSort przy częściowo posortowanych danych, tzn. dla 25%, 50%, 75% oraz 95%. Gorzej jednak radzi sobie przy posortowanych 99% oraz 99.7%, gdzie złożoność czasowa okazała się największa, zaś najmniejsza dla sortowania przez scalanie (MergeSort).
- Dobór algorytmu warunkuje czas, w jakim zbiory danych zostaną posortowane, przy czym MergeSort przez swoją stabilność i relatywnie niewielkie rozrzuty wyników sprawia, że może okazać się dobrym wyborem przy niewielkich rozmiarach danych, podobnie do sortowania introspektywnego (IntroSort), które dostarcza nam dodatkowo mechanizmy zabezpieczające przed uzyskaniem złożoności obliczeniowej $O(n^2)$, szczególnie wydłużającej czas wykonania mechanizmu przy dużych zbiorach danych.
- Warto także zwrócić uwagę na sposób, w jaki elementy są dzielone na mniejsze podzbiory w każdym z wykorzystanych algorytmów. Szczególnie uwidoczniają to różnice, kształtujące się między QuickSortem, który bazuje na algorytmie 3-way partitioning a IntroSortem, opartym o algorytm Lomuto.

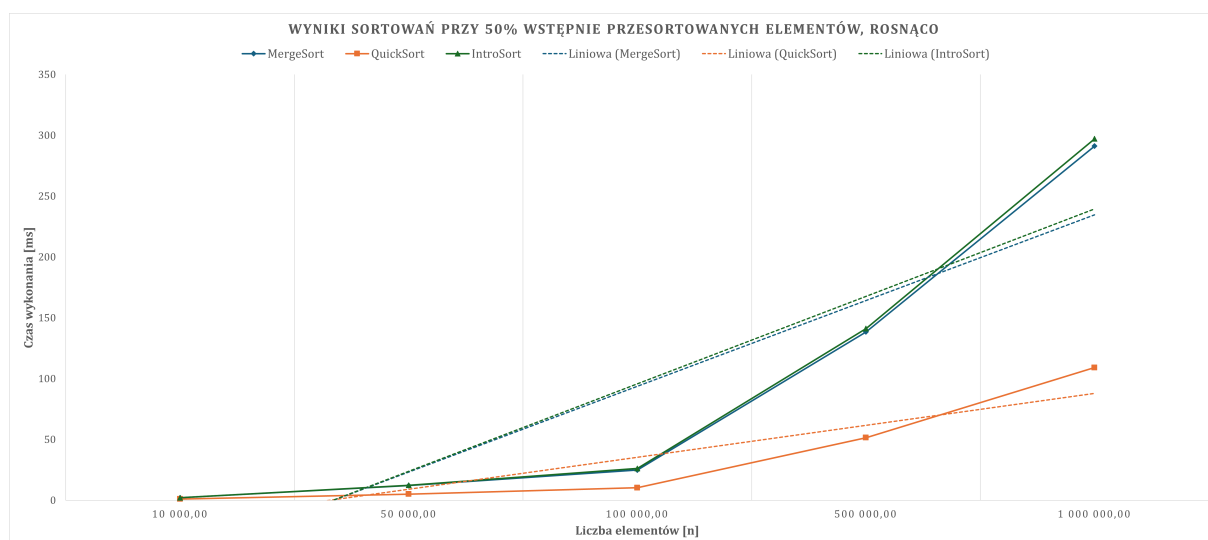
4 Wykresy



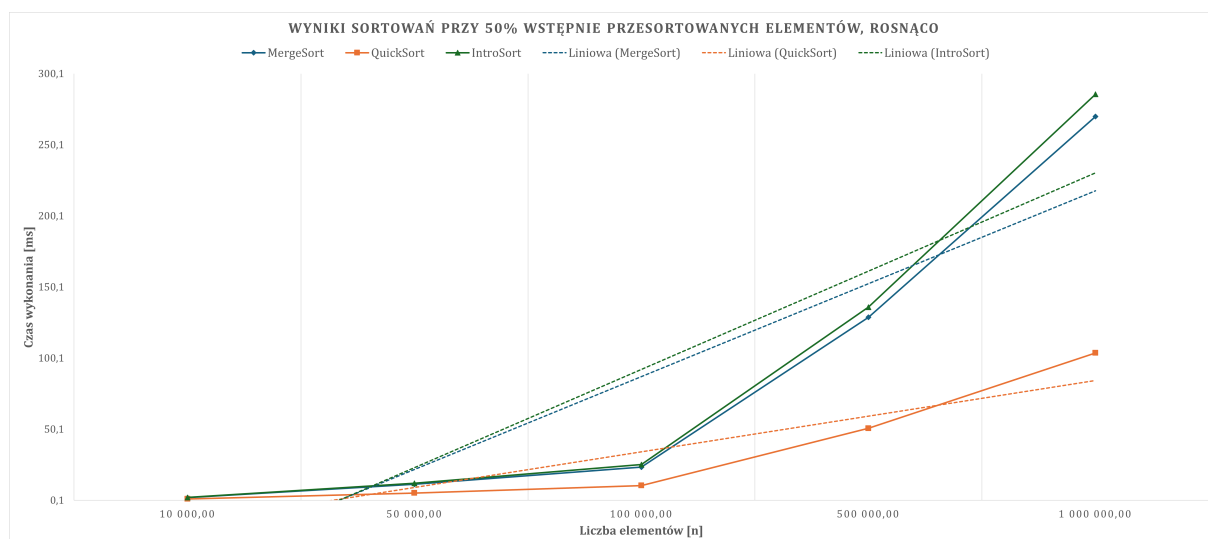
Rysunek 8: Zestawienie sortowań dla 0% wstępnie posortowanych elementów, rosnąco



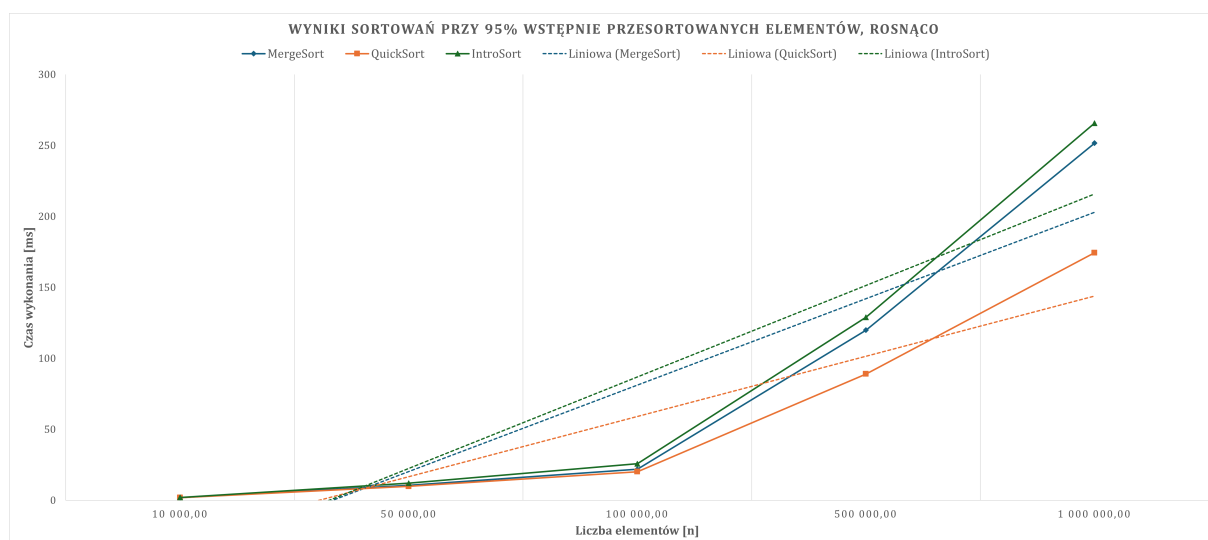
Rysunek 9: Zestawienie sortowań dla 25% wstępnie posortowanych elementów, rosnąco



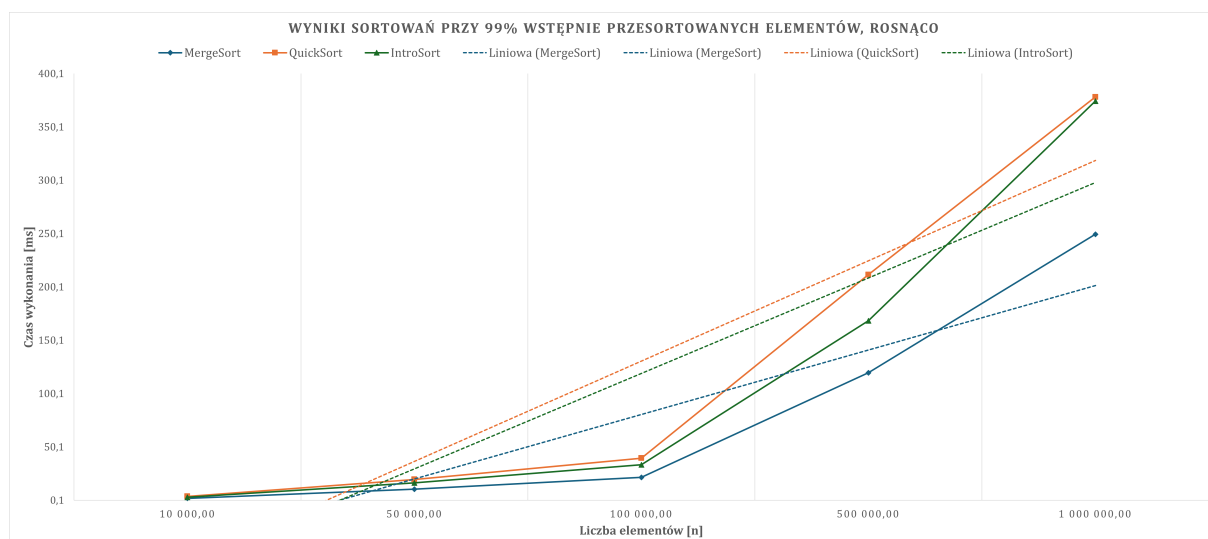
Rysunek 10: Zestawienie sortowań dla 50% wstępnie posortowanych elementów, rosnąco



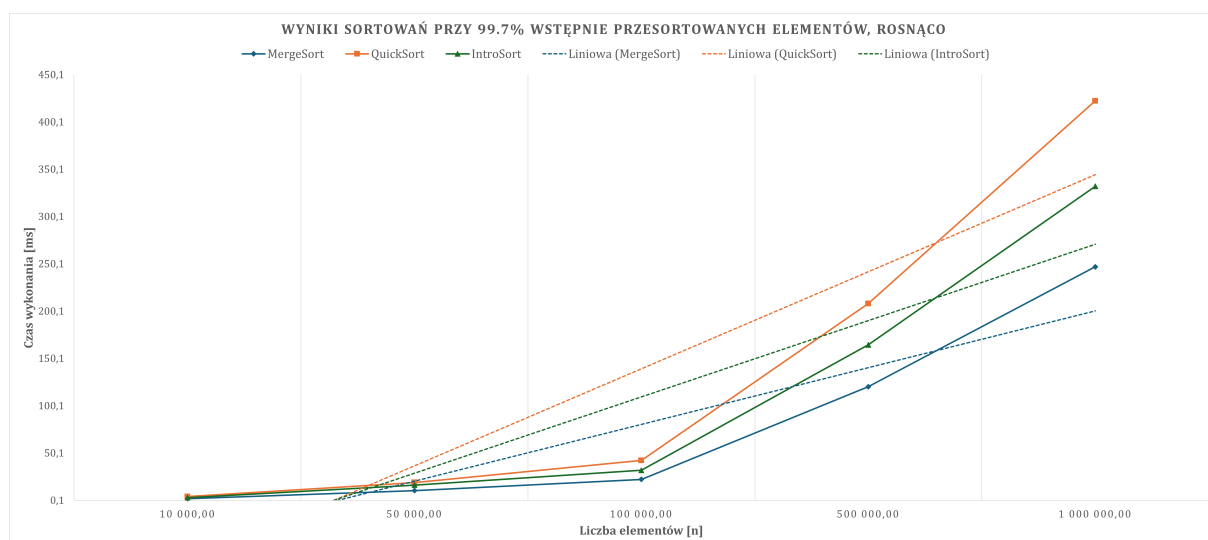
Rysunek 11: Zestawienie sortowań dla 75% wstępnie posortowanych elementów, rosnąco



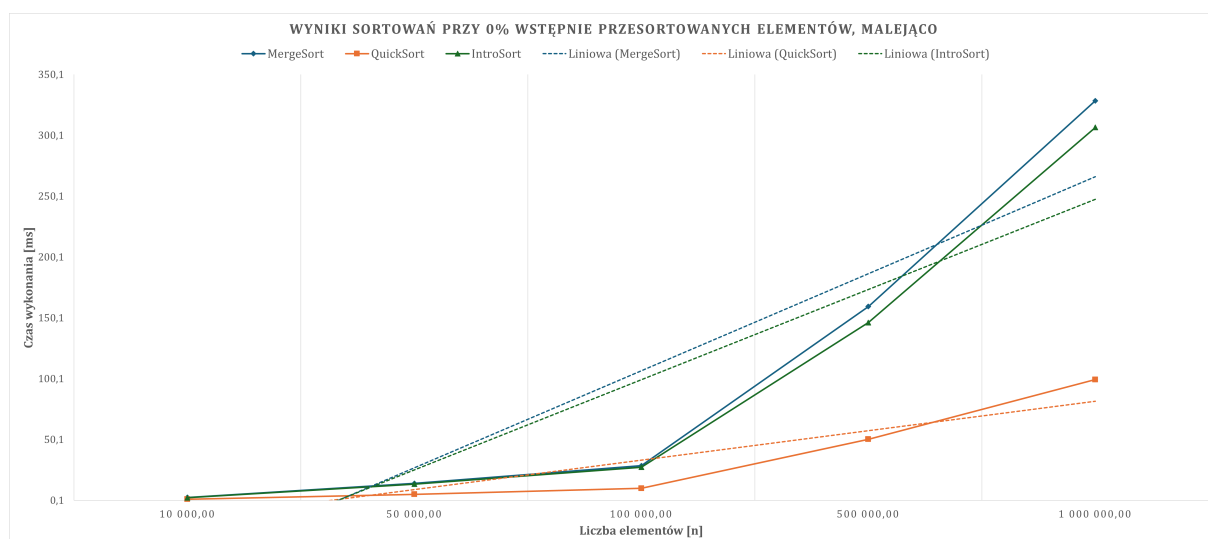
Rysunek 12: Zestawienie sortowań dla 95% wstępnie posortowanych elementów, rosnąco



Rysunek 13: Zestawienie sortowań dla 99% wstępnie posortowanych elementów, rosnąco



Rysunek 14: Zestawienie sortowań dla 99.7% wstępnie posortowanych elementów, rosnąco



Rysunek 15: Zestawienie sortowań dla 0% wstępnie posortowanych elementów, malejąco

5 Literatura

- M.T. Goodrich, R. Tamassia, D. Mount, *Data Structures and Algorithms in C++*, 2nd ed., John Wiley & Sons, Inc., Hoboken, NJ, USA, 2011
- M. T. Goodrich, R. Tamassia, *Algorithm Design and Applications*, 1st ed., Wiley, 2014, ISBN: 978-1-118-33591-8.
- GeeksforGeeks, *Quick Sort*, <https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>, data dostępu: 29.04.2026 r.
- GeeksforGeeks, *Lomuto Partition Algorithm*, <https://www.geeksforgeeks.org/dsa/lomuto-partition-algorithm/>, data dostępu: 29.04.2026 r.
- pineapples (DEV Community), *Writing a 'Median of Three' Pivot Helper for QuickSort*, <https://dev.to/pineapples/writing-a-median-of-three-pivot-helper-for-quicksort-289m>, data dostępu: 29.04.2026 r.
- Big-O Cheat Sheet, *Big-O Complexity Chart*, <https://www.bigocheatsheet.com/>, data dostępu: 29.04.2026 r.